

Checklist base de buenas prácticas

ID	Criterio por evaluar	Descripción	Fundamento	¿Qué problema ayuda a evitar?
C-01	Abstracción de entidades	Cada clase representa un elemento o concepto real del negocio que es necesario para el sistema. No se crean clases para representar datos simples, cálculos o acciones que no necesitan ser objetos.	Identificar objetos del mundo real, tener abstracciones consistentes y eliminar clases innecesarias.	Clases que no representan conceptos reales del negocio o que no tienen una razón para existir.
C-02	Cohesión de las clases	Los atributos y métodos de una clase deben estar relacionados con una misma responsabilidad o propósito.	Las clases deben tener un único propósito central.	Clases que realizan tareas diferentes y son difíciles de mantener.
C-03	Acoplamiento entre componentes	Las clases deben depender de otras únicamente cuando sea necesario, evitando dependencias innecesarias o circulares.	Mantener un acoplamiento bajo, de manera que las clases puedan modificarse sin afectar	Que un cambio en una clase obligue a modificar muchas otras.

			innecesariamente a otros componentes.	
C-04	Distribución de responsabilidades	Cada clase debe realizar las operaciones que corresponden a sus datos y responsabilidades. Las tareas que coordinan varias clases deben asignarse al componente adecuado.	La abstracción de cada clase debe ser clara y sus operaciones deben corresponder con la responsabilidad que representa.	Reglas de negocio dispersas o clases que realizan tareas que corresponden a otras.
C-05	Encapsulamiento	Los atributos deben protegerse para evitar modificaciones directas desde otras clases. Cada clase debe controlar sus propios datos y mantener sus reglas.	Ocultar los detalles internos de implementación y controlar el acceso a los datos mediante operaciones definidas por la propia clase.	Modificaciones indebidas y objetos con información en estados incorrectos.
C-06	Claridad de los nombres	Los nombres describen el concepto del negocio o lo que hace la operación y siguen una convención única	Utilizar nombres claros, descriptivos y consistentes que permitan comprender la función de cada elemento sin necesitar explicaciones adicionales.	Nombres confusos, ambiguos o utilizados con diferentes convenciones.
C-07	Diseño de interfaces	Las clases deben ofrecer métodos que permitan realizar operaciones relacionadas con su función en el negocio, en lugar	Las interfaces deben ofrecer servicios coherentes con la abstracción de la clase y proporcionar operaciones	Clases que no ofrecen operaciones suficientes para cumplir sus responsabilidades.

		de limitarse a consultar o modificar atributos.	relacionadas cuando sean necesarias, incluyendo acciones opuestas.	
C-08	Organización de clases y paquetes	Las clases deben agruparse de acuerdo con sus funciones y responsabilidades, siguiendo la arquitectura establecida (en este caso arquitectura de capas).	Organizar el software en niveles y bloques bien definidos, agrupando los elementos que tienen responsabilidades relacionadas.	Paquetes desorganizados que mezclan clases sin relación entre sí.
C-09	Manejo de dependencias	Las dependencias entre paquetes y capas deben seguir una dirección definida. Las clases del dominio no deben depender de componentes externos o de capas superiores.	Mantener una dirección de dependencias coherente con la arquitectura, evitando que las capas de mayor nivel dependan incorrectamente de las inferiores o viceversa.	Dependencias circulares y capas que dependen incorrectamente unas de otras.
C-10	Simplicidad del diseño	Cada clase, atributo, método o capa debe tener una utilidad justificada por los requisitos. Se deben evitar elementos innecesarios o complejidad anticipada.	Diseñar únicamente los elementos necesarios para resolver los problemas actuales del sistema, evitando agregar funcionalidades o estructuras que todavía no se requieren.	Complejidad innecesaria y componentes que no aportan funcionalidad.
C-11	Facilidad de mantenimiento	Los elementos que probablemente cambien deben estar organizados de manera	Identificar los elementos que pueden cambiar y aislar sus detalles para	Tener que modificar muchas clases para

		que sus modificaciones afecten la menor cantidad posible de clases.	evitar que las modificaciones se propaguen por todo el sistema.	realizar un mismo cambio.
C-12	Evitar duplicación	Cada dato y regla de negocio debe definirse en un solo lugar, evitando repetir información o lógica innecesariamente.	Evitar código duplicado y centralizar la información y las operaciones que representan una misma regla del sistema.	Información inconsistente y reglas de negocio repetidas que pueden comportarse de manera diferente.
C-13	Separación de responsabilidades entre capas	Cada capa debe cumplir una función específica. Los controladores gestionan las solicitudes, los servicios coordinan operaciones, el dominio contiene las reglas del negocio y los DAO gestionan el acceso a datos.	Separar las responsabilidades de cada capa y evitar que un componente realice tareas que corresponden a otro. La organización debe respetar la arquitectura establecida.	Mezclar reglas de negocio, control de solicitudes y acceso a datos.
C-14	Correspondencia entre requisitos y diseño	Cada requisito debe estar representado por una o varias clases, métodos o componentes del diseño. Asimismo, cada elemento del diseño debe tener una razón que lo justifique.	Mantener la correspondencia entre los requisitos y los elementos del diseño, asegurando que la arquitectura contemple las necesidades del sistema.	Requisitos sin implementar o elementos del diseño que no responden a una necesidad.
C-15	Evitar almacenar datos calculables	No se deben guardar como atributos valores que pueden	Evitar almacenar información redundante y	Guardar información repetida que puede

		obtenerse a partir de otros datos, salvo que exista una justificación para conservarlos.	mantener los datos derivados actualizados mediante su cálculo a partir de la información original.	quedar desactualizada o contradecir otros datos
C-16	Tipos de datos apropiados	Cada atributo debe utilizar un tipo de dato apropiado para la información que representa.	Seleccionar tipos de datos que representen correctamente la información y evitar utilizar tipos primitivos cuando se necesita una representación más específica	Cada atributo debe utilizar un tipo de dato apropiado para la información que representa.